ProdRAG-Lab —— 面向企业知识库的“可评测、可治理”的问答系统（RAG + LoRA + Agent + RL）

ProdRAG-Lab 是一个面向生产环境的 RAG 实验与评测平台，用于系统性验证检索、微调与 Agent 策略对可靠性的影响。

代码块

```
conda activate prodrag
cd prodrag-lab
```

项目思路

A. RAG：把“能跑”升级成“可控 + 可评测”

你要做的不是再搭一次RAG，而是做RAG评测与治理。

功能点（必做）

- **语料处理**：清洗、去重、分块策略对比（fixed-size / recursive / markdown heading）
- **索引**：BM25 + dense（双路召回）
- **重排**：cross-encoder 或 LLM rerank（先实现轻量 cross-encoder 优先）
- **引用约束生成**：回答必须带引用（chunk id / url / offset）
- **拒答策略**：召回分数阈值 + 证据覆盖度阈值 + “不知道”模板

评测体系（大厂非常吃这一套）

- 构建一个 **RAGEval 数据集**（自动生成 + 人工抽检）
  - 问题类型分桶：事实、人物关系、时间线、同名消歧、跨文档整合、不可回答
- 指标（至少做 6 个）：
  - **Answer EM/F1**（能算就算）
  - **Faithfulness/Attribution**：回答中的关键断言是否能被引用片段支持
  - **Context precision/recall**：检索出来的片段是否包含正确证据
  - **拒答准确率**：不可回答时是否正确拒答

- **延迟**：p50/p95
- **成本**：token 与调用次数

> 面试官问你“你RAG做得怎么样”，你直接用 **指标表 + ablation**，瞬间从“玩具”变“工程项目”。

B. SFT/PEFT：别再训练“百科问答”，训练“RAG助手的行为”

你现在 LoRA 微调如果只是“喂知识”，面试官会觉得“那不就是背书”。
更强的是：**微调模型学会 RAG 的格式与工具使用**。

SFT 数据要长这样：

- 输入：用户问题 + 检索到的上下文（带 chunk id）
- 输出：**严格格式**：
  - `analysis`（可选，不给也行）
  - `answer`
  - `citations: [chunk_id...]`
  - `refusal`（当不可回答时必须触发）

你要做的对比实验：

- Base LLM + RAG（无微调）
- LoRA-SFT + RAG（学会引用、拒答、少胡说）
- 指标对比：faithfulness↑、拒答↑、幻觉率↓

这就是“微调服务于产品行为”，而不是“微调让我更懂HP”。

C. Agent：让它“会查证 + 会多跳检索”

Agent不要做成“花里胡哨的自动上网”，大厂更喜欢 **任务边界清晰** 的 agent。

最实用的 Agent 形态：Retriever-Agent

- Step1：解析意图（需要单跳还是多跳？是否消歧？）
- Step2：多查询改写（query rewrite）并检索
- Step3：冲突检测（不同chunk矛盾）
- Step4：再检索补证据（evidence completion）
- Step5：生成带引用的最终答案，否则拒答

评测 agent 的点

- 多跳问题的正确率提升
- 冲突问题“不会乱选一边”而是说明不确定/需要更多证据

D. 强化学习：用 DPO/ORPO 把“引用正确、拒答正确”优化出来

你不一定要上 PPO（工程成本高、容易翻车）。
面试最稳的路线：DPO/ORPO（偏好优化），然后再加一个“PPO可选”。

怎么做偏好数据（可落地）

- 对同一问题，生成两个答案 A/B：
  - A：引用正确、证据足、拒答正确
  - B：引用乱、证据不足、答非所问或幻觉
- 用规则 + LLM judge（或你自己规则）自动打偏好对
- 用 DPO/ORPO 训练一个 LoRA 头（轻量）

你可以宣称的成果

- Faithfulness 提升 X%
- Refusal accuracy 提升 X%
- “unsupported claims” 比例下降 X%

> 强化学习在面试里最怕“我做了PPO但没法证明好”。DPO这种更容易做出**稳定可展示**的对比结果。

A. RAG：把“能跑”升级成“可控 + 可评测”

1 数据准备与预处理

多语种rag：英文主语料 + 中文问答增强

使用py爬取医学+大模型方面的知识库

- 英文：爬取axive相关医学大模型论文和资料

运行 `scripts/fetch_arxiv.py`

代码块

```
python scripts/fetch_arxiv.py \
  --run_id 2026-01-27_run001 \
  --max_papers 400 \
  --query '(cat:cs.CL OR cat:cs.LG) AND (all:"large language model" OR all:LLM) AND (all:medical OR all:clinical OR all:healthcare)' \
```

```
  --query '(cat:cs.CV OR cat:eess.IV) AND (all:multimodal OR all:"vision-language" OR all:"image-text") AND (all:medical OR all:clinical OR all:radiology)' \
  --query '(cat:q-bio.QM OR cat:q-bio.NC OR cat:q-bio.OT) AND (all:multimodal OR all:"large language model" OR all:LLM) AND (all:medical OR all:clinical)'
```

- 中文：爬取最近5-6年国家级官方机构（国家卫健委（NHC）等等）的文档和医学学会 / 协会发布的指南还有知网论文

没有提供统一接口只能手动下载

对齐raw data 的介绍表

编辑并运行，对齐数据介绍表

`scripts/build_meta_for_pdfs.py`

格式如下：

代码块

```
            rec = {
                "doc_id": f"pdf:{sha[:16]}",
                "source": args.source,

                # arxiv-specific id doesn't exist for local pdfs
                "arxiv_id": None,

                "title": title,
                "authors": authors,

                "published": published,
                "updated": updated,

                "summary": summary,

                "query": args.query or "",

                # no canonical web urls for local PDFs
                "url_abs": None,
                "url_pdf": None,

                # keep same meaning as arxiv meta: where the PDF lives locally
                "download_path": str(pdf.as_posix()),
                "meta_path": str(meta_path.as_posix()),

                "sha256": sha,
                "status": status,
```

```
                "fetched_at": utc_now_iso(),

                # --- extra fields (safe to keep; doesn't break arxiv-style consumers if they ignore unknown keys) ---
                "language": lang,
                "page_count": basic.get("page_count"),
                "pdf_metadata": meta,
            }
```

会生成 `data/raw/zh/meta` 和 `data/raw/zh/manifest.jsonl`

建立数据库

把 PDF 变成“可检索的知识库”

- 能被模型理解的文本
- 能被向量检索的碎片（chunks）
- 能快速近似检索的索引（index）

raw(PDF) → interim(text) → processed(chunks) → index，bm25

运行

代码块

```
python src/rag/build_kb.py configs/build.yaml
```

1. 首先抽取（extract）PDF → 每篇一个 text-json，输出目录 `data/interim/text`
2. 然后分块（chunk）text-json → `chunks.jsonl`，输出目录 `data/processed/chunks/chunks.jsonl`
3. 最后建索引（index）chunks.jsonl → FAISS + docstore + meta，分为语义检索索引和关键词检索索引

语义检索索引：中英文数据分别按照文件夹分类的形式区别，然后分别使用 `BAAI/bge-small-en-v1.5` 和 `BAAI/bge-small-zh-v1.5` 建立中英文FAISS库 `data/index/faiss_en`、`data/index/faiss_zh`

> 语义检索索引（Dense / Semantic Retrieval）是什么？
>
> 使用 `BAAI/bge-small-en-v1.5` 和 `bge-small-zh-v1.5` 分词向量化
>
> FAISS = Facebook AI Similarity Search
>
> 给一堆向量建“索引”，让你能在百万级向量里毫秒级找最近的那些

> 分词器解决的是符号层表示问题，而向量化关注的是语义空间建模；FAISS 则是对语义向量进行高效相似度检索，它们在系统中处于不同层级。
>
> Dense 的优点
>
> - 不要求关键词完全匹配：同义改写、不同表述也能命中
> - 能跨句式、跨表达方式找相似内容
>
> Dense 的缺点
>
> - 对“稀有术语/缩写/数字/符号”不如关键词检索稳
> 例：`HBsAg`、`CD4+`、`10^5 copies/mL`、`p24` 这种，BM25 常常更强
> - 模型有时会“语义漂移”（看着像相关，其实不是）

▲ 如果你用英文模型去编码中文，常见现象就是：**召回相关性显著下降**。

关键词检索索引：

> 关键词检索索引（Sparse / Lexical Retrieval）是什么？
>
> Sparse 的优点
>
> - 对专有名词、缩写、数字、精确匹配很强
> - 可解释性更强（命中词项越多越高）
>
> Sparse 的缺点
>
> - 同义改写不友好：你没写那个词，就很难命中
> - 中文分词质量会影响效果（jieba 不是完美，但够用了）

目的：为了下面 retrieval 双路索引方便而设计**Hybrid 双路**：BM25（关键词） + FAISS（语义）两路召回

2 大模型基座

▲ 大模型基座不是“选一个模型写死”，而是**可插拔、可切换、可对比**的设计，服务于后续的 **RAG / LoRA / Agent / 强化学习** 等实验。

核心原则是三点：

1. **接口统一**：上层 RAG、评测、Agent 不感知底层模型来源
2. **线上/本地解耦**：开发与评测阶段可用 API，训练与可控实验用本地
3. **便于对比实验**：Base vs LoRA、API vs Local 能公平对比

API

相关代码在

deepseek api

在不引入微调的情况下，评估：

- RAG 本身的检索质量
- 引用与拒答策略是否能靠 prompt / pipeline 控制

在终端导入API key

`export DEEPSEEK_API_KEY=你的key`

`export DEEPSEEK_API_KEY=sk-32146ae3fd874e47b62be2505c5c9710`

本地大模型1（生成效果一般）

DeepSeek-R1-Distill-Qwen-1.5B

选择原因（面试重点）

1. **参数规模适中（1.5B）**
   - 单卡（如 4070 / A10）即可完成 LoRA 微调
   - 工程可复现，不依赖大算力
2. **蒸馏自强推理模型**
   - 相比普通 Qwen，小模型具备更好的 reasoning 行为
   - 更适合训练“RAG 助手行为”（引用、拒答、格式）
3. **生态友好**
   - HuggingFace + LLaMA-Factory 原生支持
   - 易于集成 LoRA / DPO / ORPO

> ▲ 我的项目在大模型层采用 API + 本地模型的可插拔设计。
>
> API 模型（DeepSeek）作为强基线，用于验证 RAG 系统本身；
>
> 本地模型（DeepSeek-R1-Distill-Qwen-1.5B）用于 LoRA 和偏好优化，训练模型学会引用、拒答和稳定格式。
>
> 上层 RAG、Agent 和评测完全复用，从而公平对比 Base+RAG 和 LoRA+RAG 的行为差异。

- **如何下载：**

修改 HuggingFace 的镜像源

```
export HF_ENDPOINT=https://hf-mirror.com
```

- 修改模型下载的默认位置

```
代码块
export HF_HOME=/home/zps/models/llm
```

- 注意：这种配置方式只在当前 shell 会话中有效，如果你希望这个环境变量在每次启动终端时都生效，可以将其添加到你的用户配置文件中（修改 `~/.bashrc` 或 `~/.zshrc`）
- 检查环境变量是否生效

```
代码块
echo $HF_ENDPOINT
echo $HF_HOME
```

docker里面也要单独挂载 `export HF_HOME=/workspace/HFModels`

辅助模型

中文 embedding

```
代码块
hf download BAAI/bge-small-zh-v1.5 \
  --local-dir /root/autodl-tmp/models/bge-small-zh-v1.5
```

英文 embedding

```
代码块
hf download BAAI/bge-small-en-v1.5 \
  --local-dir /root/autodl-tmp/models/bge-small-en-v1.5
```

---

reranker

```
代码块
```

```
hf download BAAI/bge-reranker-base \
  --local-dir /root/autodl-tmp/models/bge-reranker-base
```

下载模型到HF_HOME=/home/zps/models/llm

HF 缓存统一指到 /root/autodl-tmp/models

代码块

```

export HF_ENDPOINT=https://hf-mirror.com
export HF_HOME=/root/autodl-tmp/models
export TRANSFORMERS_CACHE=/root/autodl-tmp/models/transformers
export HF_HUB_CACHE=/root/autodl-tmp/models/hub
export SENTENCE_TRANSFORMERS_HOME=/root/autodl-tmp/models/sentence-transformers
```

本地的模型2

models--unsloth--DeepSeek-R1-Distill-Qwen-7B-bnb-4bit

用 4bit 量化好的 7B Instruct 模型

✅ 已经 4bit 量化

✅ 体积 ≈ 4~5GB

✅ 不用自己量化

✅ 4070 8GB 稳跑

✅ 和原版 R1 7B 同能力

✅ 下载稳定

运行下面代码

代码块

```
import re
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_ID = "unsloth/DeepSeek-R1-Distill-Qwen-7B-bnb-4bit"

def strip_think(text: str) -> str:
    # 去掉 <think>...</think>，避免污染评测
    return re.sub(r"<think>.*?</think>", "", text, flags=re.S).strip()

def main():
    tok = AutoTokenizer.from_pretrained(
```

```
        MODEL_ID,
        trust_remote_code=True,
        use_fast=True,
    )

    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID,
        trust_remote_code=True,
        device_map="auto",
        torch_dtype=torch.bfloat16,  # 4070 支持 bf16；如果报错就删掉这一行
    )
    model.eval()

    messages = [
        {"role": "user", "content": "用一句话介绍你是谁，并说明你是否会输出<think>标签。"}
    ]

    try:
        text = tok.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
    except Exception:
        # fallback：没有模板时简单拼接
        text = messages[-1]["content"]

    inputs = tok(text, return_tensors="pt").to(model.device)

    with torch.no_grad():
        out = model.generate(
            **inputs,
            max_new_tokens=200,
            do_sample=False,           # 评测建议关采样，稳定可复现
            repetition_penalty=1.05,
        )

    ans = tok.decode(out[0], skip_special_tokens=True)
    print("\n=== RAW ===\n", ans)
    print("\n=== STRIPPED ===\n", strip_think(ans))

if __name__ == "__main__":
    main()

```

▲ 在基线阶段我使用 1.5B 本地模型做可控性验证，但在实际评测中发现其在表格抽取与数值比较任务上能力不足，因此升级到 7B 4bit 量化版本，在显存受限条件下实现性能与资源的平衡，同时保证本地可复现与低成本部署。

为什么 1.5B 不够?

在 RAG 评测中，很多问题涉及表格抽取、数值比较、跨 chunk 归纳。 1.5B 模型在这类结构化信息理解任务上表现明显不足，容易出现“看到证据却无法正确抽取数值”的情况。这会影响基线的有效性，因为我们无法区分是检索链路问题还是模型能力瓶颈。

为什么不直接用 API?

API 模型能力更强，但存在两个问题：

成本不可控（大规模评测）；无法做可控的 LoRA 微调对比实验

我希望构建一个“可评测、可治理”的系统，因此必须有可本地控制的模型基线。

为什么选 7B 4bit?

我使用 RTX 4070 8GB 显卡，本地无法加载 BF16 7B（需要 14GB+）。

因此选择 4bit 量化版本，在显存占用约 5~6GB 的情况下实现 7B 能力。

实测结果表明：

- 表格理解能力显著提升
- 数值比较准确率提高
- 引用稳定性增强

同时仍保持本地推理可复现性。

3 简单rag

检索层 retrieval

`src/rag/retrieval.py`

流程

- 用户输入问题
- 系统实时把问题向量化（embedding）
- Dense：从向量库拿 topK（比如 30）
- BM25：从 BM25 拿 topK（比如 30）
- 合并去重（比如最多 60）

- reranker 精排（比如取 8）*支持可选*
- 丢给大模型生成答案

信息检索

传统方法1980s：TF-IDF = Term Frequency × Inverse Document Frequency（词频 × 逆文档频率）

一个词在某篇文档中有多重要？在这篇文章里出现多 → 重要；但如果在所有文章里都常见 → 就没那么重要

- BM25

解决两个关键问题：

词频饱和：词出现 10 次和 100 次差别不该太大

文档长度归一化：长文档不该天然占优

- 向量检索（Sentence-Transformers + FAISS）

把文本变成向量（Embedding）；用 FAISS 建一个“向量数据库”。

使用方法

中英文双语混合

Sparse：先 BM25 取一批候选（召回关键词强）

Dense：再用向量相似度补充候选（召回语义强）

最后用 reranker（如 bge-reranker）重排

生成层 pipeline

—— 强制 citations + refusal 阈值（把“禁止无法回答”删掉）

`src/rag/pipeline.py`

首先大模型翻译器翻译query，检索top_chunks

然后建立prompt，传给大模型（可插拔的本地/API大模型）

流式生成结果

完整rag

代码块

```
python src/rag/cli_chat.py configs/rag.yaml
```

流式输出，一次加载，多次问答

拒答逻辑：

- forced_refusal（系统强制拒答）

来源：检索置信度（通常用 reranker top1 的分数），如果最相关证据的置信度不够（top1_prob 太低），系统就强制拒答

4 离线测评

- [x] ~~chunk→dense/faiss + bm25→融合召回→rerank→拼 prompt→LLM~~
- [x] ~~完成双路召回（BM25 + dense）+ 重排~~
- [x] ~~强制引用 + 拒答阈值~~
- [x] ~~FastAPI：流式输出、请求ID、日志~~
- [ ] 离线评测脚本：至少 200 条 QA（自动生成+抽检）
- [ ] 输出一张对比表：baseline vs +rerank vs +refusal

离线评测集生成

运行 `src/eval/build_evalset.py` 可以生成250条数据并且过滤到200条

代码块

```
mkdir -p data/eval_rag
python src/eval_rag/build_evalset.py \
  --llm_cfg configs/rag.yaml \
  --chunks data/processed/chunks/chunks.jsonl \
  --n_raw 250 \
  --n_keep 200
```

每条样本：question+gold_answer+gold_citations

这样你后面才能对 RAG 做**可量化评测**：

- 检索：Recall@k / MRR / nDCG（看能不能把 gold_citations 检索出来）
- 生成：看生成答案是否覆盖 gold_answer（可做 LLM-as-judge 或规则/相似度评分）

> 🔖 我把‘检索评测’和‘生成评测’解耦了：先有可追溯的 gold evidence（citations），才谈得上检索指标；否则评测是飘的。

方法：

1. **读入 chunks.jsonl**（你的离线语料切分结果）

2. **chunk 规范化 + 粗过滤**（短文本/参考文献块剔除）
3. **读取 rag.yaml 并“适配” build_llm 的配置结构**（避免 KeyError: api_key）
4. **抽一个候选池 pool**（避免全量 3-4 万 chunk 全跑）
5. **循环调用 LLM 生成 QA + JSON 抽取 + 质量过滤 + 去重**
6. **先落一份 raw（比如 250）再随机打乱保留 200，并提示人工抽检**

过滤规则

chunk 内容太短（<200 字符）直接不生成

明显参考文献/DOI 堆砌（命中≥2）跳过

生成的 QA：

- question 太短/太长过滤
- answer 太短/太长过滤
- difficulty 不在 easy/mid/hard 过滤
- citations 必须且只能是当前 chunk_id（防乱引）
- 答案出现“资料不足/无法确定”等拒答倾向过滤

📜 “我为 RAG 做离线评测时，先从 processed chunks 构建一个 QA evalset。脚本会先把 chunks.jsonl 里的字段做规范化，并过滤掉过短和参考文献/doi 类低信息 chunk，避免生成空泛问题。然后读取 rag.yaml 并适配成 build_llm 期望的结构，同时优先从环境变量注入 API key，确保安全和可部署。

在生成阶段我先抽一个候选池避免全量 chunk 请求导致成本爆炸，再轮询每个 chunk 让 LLM 生成 1 条 QA，并强制 citations 只能是当前 chunk_id，这样每条样本都有明确的 gold evidence，检索可以算 Recall@k、MRR。为保证数据集质量，我对 LLM 输出做了稳健 JSON 抽取，并在规则层过滤掉过短/过长、difficulty 非法、引用不匹配、拒答式回答，同时做问题去重。最终先保留一份 raw 用于回溯，再随机抽样得到 v1_200 评测集。最后我会抽检 20-30 条，确认可回答性和忠实性，再迭代过滤规则。”

📜 标准答案（ground truth）到底从哪来？为什么它就一定正确？别的答案为什么要拿它对比？

我把“标准答案”定义为**在语料中可被证据支持、可追溯、可复核的答案**。
不是因为模型说它对，而是因为 chunk 原文能证明它对。

> A. **“引用型标准答案”**（RAG最推荐）
>
> **标准答案 = 从某个 chunk 里能直接找到的、可引用的答案**

> 生成 QA 时，LLM 不仅输出 `question/answer`，还必须输出：
>
> `support_chunk_ids`（支持证据 chunk 列表）
>
> `evidence_spans`（原文中哪几句/哪几行支撑这个答案）
>
> 这样，“正确”不是因为 LLM 说了算，而是因为：
>
> **答案能被证据 chunk 的原文逐字或近似复述验证**
>
> 并且证据可追溯（你能点开 chunk 看）
>
> **优点**：最适合做 RAG 检索评估；解释性强；可抽检。
> **缺点**：更偏“抽取式/忠实复述”，不太适合开放式推理题。

> 你这套流程里最合理的“标准答案”，就是这种：**答案 + 引用 chunk + 原文证据**。

> B. **“弱监督标准答案”**（LLM 生成，但强约束）
>
> 标准答案仍然是 LLM 生成的，但你用规则把它“压实”：
>
> 要求输出结构化 JSON（你已经在做）
>
> 质量过滤（你已经在做）
>
> 再加两步会更“像标准答案”：
>
> **证据一致性检查**：答案中的关键实体/数字/定义必须在证据 chunk 出现
>
> **反向验证**：再调用一次 LLM，给它 chunk+答案，让它判断“是否被原文支持”，不支持就丢
>
> **优点**：成本低、产量高。
> **缺点**：仍可能夹杂“幻觉”，需要抽检兜底。
>
> C. **“人工标注标准答案”**（最强，但最贵）
>
> 你只对一小部分（比如 50~200 条）做人工写答案/确认答案
>
> 这部分成为**真正的 gold set**
>
> 其余 2000 条可以当训练/开发用的“银标”
>
> **优点**：真正意义上的 ground truth。
> **缺点**：贵、慢。

对比测评实验

评估“RAG Pipeline 结构改进”（baseline → +rerank → +refusal）

代码块

```
python src/eval_rag/eval_run.py --config configs/eval/baseline.yaml
```

> **baseline**：只做“召回”，证据排序与噪声控制一般，靠运气找到好 chunk
>
> **+rerank**：在召回后做“精排”，把最能回答问题的证据放前面，回答更稳
>
> **+refusal**：再加“门控拒答”，证据不足就不答，最大化减少胡编乱造

🏫 我这个 RAG 系统做了**可复现的离线评测**：
用 200 条从文档自动生成的 QA，对比了 baseline、rerank、refusal 三种配置，从 **检索召回、证据引用合规、拒答安全性** 三个维度量化评估， rerank 提升了 MRR 和 Recall，refusal 在证据不足场景下有效抑制错误回答，整体设计是可上线、可回归测试的。

代码块

```
python src/eval_rag/eval_metrics.py \
  --runs runs/baseline.jsonl runs/rerank.jsonl runs/refusal.jsonl \
  --out_md tables/compare_rag.md \
  --out_csv tables/compare_rag.csv

```

Setting	N	Recall@10	MRR@10	CitePrec	RefusalRate	AvgLatency(ms)
baseline	200	0.140	0.081	0.824	0.900	11175.636
+rerank	200	0.665	0.541	0.965	0.440	11498.262
+refusal	200	0.665	0.541	0.964	0.470	10928.504
改进1	200	0.673	0.538	0.955	0.457	13111.748

📜 结果分析

rerank 是这个 RAG 系统里“不可替代的关键组件”。

baseline 只有 **14%**

- 说明：**粗召回+融合** 对你这套 QA **严重不够**

+rerank 直接提升到 **66.5%**

- **提升了将近 5 倍**
- 这是 cross-encoder 精排**典型、合理、可信的增益幅度**

+refusal **不影响 Recall**

- 说明 refusal 是**门控策略**，不改检索本身

rerank 不只是“把它找回来”，而是把它排到前面。

baseline MRR 很低
→ gold chunk **常常排在很靠后**

rerank 把 MRR 提升到 **0.54**

- 意味着：**gold 证据平均排在前 2 名左右**

refusal 不影响 MRR
→ 再次验证：**拒答是后处理，不破坏排序**

rerank 不仅提升了召回，还显著提高了回答的 groundedness。

关于CitePrec

baseline 已经不低（0.82）

- 说明你的 prompt + 强制引用 **是有效的**

rerank 把它提升到 **~0.965**

- 说明：
  - prompt 里噪声 chunk 更少
  - 模型更容易只引用真正相关证据

refusal **几乎不影响 citation precision**

- 这是**理想行为**

rerank 提升“可回答率”，refusal 在边界样本上收紧。

baseline **90% 拒答**

- 完全符合你之前观察到的现象
- 这是因为：
  - 没 rerank
  - gold 很多没被看到

rerank 把拒答率降到 **44%**

- 说明：
  - 有一半以上问题现在“有证据可答了”

refusal 比 rerank 多 **3% 的拒答**

- 这正是 **refusal gate 在工作**
- 它挡住的是：
  - rerank 置信度低的边界样本

refusal 在提升安全性的同时，对延迟没有负面影响，甚至略有下降。

rerank 比 baseline **多 ~300ms**

- 非常合理（cross-encoder + 多一步逻辑）

+refusal 反而更快

- 因为：
  - 一部分样本被提前拒答
  - LLM 生成路径被跳过或缩短

📜 我用同一套 200 条 QA 做了三组消融实验。

baseline 只有双路召回，Recall@10 只有 14%，说明粗召回不足以支撑高质量问答。

引入 rerank 后，Recall 提升到 66.5%，MRR 提升到 0.54，说明关键证据不仅被找回，而且排在很靠前。

同时 Citation Precision 提升到 96% 以上，验证了回答的 groundedness。

在此基础上加入 refusal，几乎不影响召回和排序，只是在低置信度样本上提高拒答率，作为安全控制。

整体延迟增加很小，符合工程预期。

5 迭代

📜 面试官：那你怎么改进？

我先看 error cases，再针对性调 topk、rerank 或阈值。

`python src/eval/export_error_cases.py`

代码块

```
(prodrag) zps@ZPS-PC2:~/prodrag-lab$ python src/eval/export_error_cases.py
[export_error_cases] written to tables/error_cases.md
retrieval_errors=67, citation_errors=5, refusal_cases=94
```

优先调 topk（扩大候选池）→ 再看 rerank_topk → 最后才调 refusal 阈值。

先让答案能找到证据（Recall），再让证据排得靠前（MRR），最后控制不确定时不乱答（Refusal）。

B. SFT/PEFT：别再训练“百科问答”，训练“RAG助手的行为”

1 环境搭建

安装 LLaMA-Factory

退出空间，新建一个SFT空间

首先安装git

`apt update`

`apt install -y git`

然后git一下

`git clone https://github.com/hiyouga/LLaMA-Factory.git`

`cd LLaMA-Factory`

`pip install -U pip`

新建一个conda环境

`conda create -n lmsft python=3.11 -y`

> 一定是 `python=3.11` 及以上

建议先单独下载torch

```
代码块
pip install torch torchvision torchaudio \
  --index-url https://download.pytorch.org/whl/cu124

```

验证是否安装成功

```
代码块
python -c "import torch; print(torch.version.cuda)"
```

然后 `pip install -e ".[metrics]"`

验证是否安装成功：

`llamafactory-cli version`

运行：

`llamafactory-cli webui`

2 训练数据生成

随机抽chunks文件夹里面的pool_size个chunk

大模型生成问题并根据所选择的chunk尝试生成 QA，

--max_tries指的是最多调用 LLM 多少次（生成尝试上限）

然后按照格式 / 抽取 / 过滤条件筛选数据

代码块

```
python src/eval/build_evalsetv2.py   --llm_cfg configs/rag.yaml   --chunks data/processed/chunks/chunks.jsonl
```

将QA 变成 SFT 数据：用 ShareGPT 风格（LLaMA-Factory 最常用、最稳）。

代码块

```
python src/sft/build_sftdata.py   --chunks data/processed/chunks/chunks.jsonl   --qa data/sft/qa_v1.jsonl   --out_dir data/sft   --val_ratio 0.10   --evidence_max_chars 1800   --limit_per_chunk 2
```

假设叫 `prodrag_sft_train.jsonl`

然后再llama-factory文件夹里dataset_info.json放入下面一段作为识别

代码块

```
  "prodrag_sft_train": {
    "file_name": "prodrag_sft_train.jsonl",
    "formatting": "sharegpt",
    "columns": {
      "messages": "conversations"
    }
  },
```

生成负样本

代码块

```
python src/sft/add_refusal_negatives.py \
  --train data/sft/train.jsonl \
  --chunks data/processed/chunks/chunks.jsonl \
  --out data/sft/train_with_neg.jsonl \
  --neg_ratio 0.15 \
  --keep_originals
python src/sft/add_refusal_negatives.py \
  --train data/sft/val.jsonl \
  --chunks data/processed/chunks/chunks.jsonl \
  --out data/sft/val_with_neg.jsonl \
  --neg_ratio 0.10 \
  --keep_originals
```

▲ 如果面试官问：

> 你 SFT 是怎么设计的？

你可以说：

> 我把检索到的 chunk 原文作为 prompt 上下文，
> 强制模型在 noisy 原始证据中做归纳比较，
> 同时通过输出格式约束保证 citation precision，
> 这样 SFT 实际上是在教模型“如何使用 RAG”，
> 而不是“背答案”。

拒答样本 0.15

3 SFT

训练配置

参考官方博客

https://llamafactory.readthedocs.io/zh-cn/latest/getting_started/webui.html

有效 batch size = batch_size × gradient_accumulation × GPU数量

训练结果

loss 起始 ~1.2 → 收敛到 ~0.02

训练2

eval效果

代码块

```
***** predict metrics *****
  predict_bleu-4                    =       9.509
```

```
  predict_model_preparation_time =     0.0031
  predict_rouge-1                =    35.6139
  predict_rouge-2                =     16.671
  predict_rouge-l                =    16.7229
  predict_runtime                = 0:06:41.05
  predict_samples_per_second     =      0.125
  predict_steps_per_second       =      0.002
[INFO|2026-02-09 21:44:59] llamafactory.train.sft.trainer:143 >> Saving prediction results to saves/DeepSeek-R1-1.5B-Distill/lora/eval_2026-02-09-16-32-52/generated_predictions.jsonl
```

4 测评

首先根据本地模型和sft模型分别进行测试集测试

代码块

```
python src/eval_sft/eval_run.py --config configs/sft/baseline.yaml
python src/eval_sft/eval_run.py --config configs/sft/sft.yaml
```

获取 `artifacts/rag_eval/baseline_local.jsonl` 和 `artifacts/rag_eval/baseline_sft.jsonl`

然后分别运行进行测评

> ▲ SFT 是“行为学习”，不是“检索能力提升”。因此需要换一套指标
>
> 业界评估 SFT / Instruction-Tuning 常用指标
>
> **生成质量指标（LLM 侧）**
>
> 1️⃣ Exact Match (EM)
>
> 是否完全匹配标准答案
>
> 2️⃣ F1
>
> token overlap
>
> 3️⃣ BLEU / ROUGE / BERTScore
>
> 语义相似度
>
> BERTScore 是现在比较流行的
>
> 4️⃣ GPT-as-a-Judge（大厂常用）
>
> 用更强模型评判：
>
> 是否回答正确

是否引用充分

是否 hallucinate

是否格式正确

现在业界大量用这种。

RAG Faithfulness 指标（你这个项目应该加这个）

这是重点。

现在业界 RAG 论文常用：

1️⃣ Faithfulness（是否基于检索内容回答）

计算：

回答中的 claim 是否能在 retrieved chunks 找到支持

或 LLM judge 判定“是否有证据支撑”

2️⃣ Attribution Accuracy

回答是否引用了正确 chunk

3️⃣ Hallucination Rate

回答中是否出现检索内容中不存在的事实

4️⃣ Context Utilization

回答是否真正使用了检索上下文

拒答能力评估（你应该强化）

你现在 RefusalRate 只是“拒答比例”。

这不够。

业界更看：

1️⃣ Refusal Precision

该拒答时是否拒答

2️⃣ Refusal Recall

该拒答没拒答算严重错误

3️⃣ Refusal F1

目标

学会按格式输出（analysis / answer / citation）

学会“该拒就拒”

学会引用支持证据

降低 hallucination

refusal_precision

refusal_recall

refusal_f1

hallucination_rate

citation_support_rate

实际使用指标：

在训练什么：
CitePrec（Citation Precision）：模型给出的引用中，有多少是真正“应该引用”的 gold citation

CitationSupport：模型引用的内容，是否真的“支持”它的回答

HallucRate（幻觉率）：模型回答中是否包含没有被引用支持的信息

代码块

```
python src/eval_sft/eval_metrics.py \
  --run sft_localv2=artifacts/rag_eval/base_localv2.jsonl \
  --out_csv tables/compare_sft_sftv2.csv
python src/eval_sft/eval_metrics.py \
  --run sft_localv2=artifacts/rag_eval/sft_localv2.jsonl \
  --out_csv tables/compare_sft_sftv2.csv



```

答案语义相似度：**EmbSimFull**

语义正确率：**EmbAccSentMax@0.70**

Setting	N	EmbSimFull	EmbSimSentMax	EmbAccSentMax@0.70	TokF1	AvgLatency(ms)
SFT	100	0.532	0.539	0.48	0.12	12846

Setting	N	Valid N	EmbSimFull	EmbSimSentMax	EmbAccSentMax@0.50	TokF1	Refusal Rate(pred)	AvgLatency(ms)
	100	100	0.42158	0.4317	0.38	0.10214	0%	13726.8

baselo cal								
sft	100	100	0.53170	0.5386	0.48	0.120262	0%	12846.9
dpo	100	100	0.53849	0.5463	0.52	0.118237	10%	11035.3

▲ **结果分析**

在实验中我们比较了 Base → SFT → DPO 三个阶段的性能。

SFT阶段带来了主要提升，因为模型学习了 evidence-based answering 的基本能力。

DPO阶段的提升相对较小，但仍然带来了：

- Embedding similarity 提升
- Answer accuracy 提升

主要原因是 DPO优化的是 **行为偏好（citation / refusal）**，而不是知识能力。

因此语义指标提升有限，但可以显著改善：拒答效果，幻觉控制

5 迭代

双语检索修复

检索失败（没命中 gold） + 不该答却答（漏拒） + 引用闭环但闭环在错误证据上

修retriver代码和pipline代码

en/zh 候选配额、防止 topK 截断把 arxiv 挤掉、双语 rerank、阈值用 sigmoid 概率、修掉 pipeline 里的 t0~t5 未定义

修eval_run代码

translated_query 彻底关掉了，双语召回逻辑根本没启用

解析 fallback 逻辑正在掩盖真实错误，模型根本没输出 citations，但自动帮它补上

拒答逻辑提升

`top1_score` 是 **reranker 的 raw logit，**而不是 sigmoid 概率。拒答几乎永远不触发

- 用“间隔（margin）”比单点阈值更稳，比起看 top1 高不高，更稳的是看：top1 是否明显强于 top2/topk
- 抽 query 的关键词（去停用词后），看 top1 chunk 覆盖了多少关键词（或 overlap token ratio），覆盖过低 ⇒ 直接拒答
- 把“解析失败”视为高风险，倾向拒答或重试

可以并行检测

观察GPU占用

```
代码块
1   nvidia-smi dmon -s u
```

单单条问题太慢，GPU显存明显富足，

eval.num_workers 并行方案的原理

你现在这个“之前的方法”本质是：在一个进程里，用线程池并行调 run_one，共享同一个 llm 和 retriever，由主线程串行写结果文件。具体拆一下：

- 1）读取配置，得到并行度

eval_cfg = cfg.get("eval", {}) or {}

num_workers = int(eval_cfg.get("num_workers", 1))

- 2）把还没跑过的样本挑出来

pending_items = [it for it in qa_items if it.get("qid") not in done_qids]

total_pending = len(pending_items)

- 3）如果 num_workers <= 1，就走原来的串行逻辑

逐条 for item in pending_items，调用 run_one(...)，然后 write_jsonl_append(...)，行为和未改代码一致，完全兼容。

- 4）如果 num_workers > 1，就启用 ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=num_workers) as ex:

  future_to_qid = {ex.submit(_process_item, it): it.get("qid") for it in pending_items}

  for fut in as_completed(future_to_qid):

    row = fut.result()

    write_jsonl_append(out_path, row)

这里的关键点：

- ThreadPoolExecutor(max_workers=num_workers)：创建一个固定大小的线程池（例如 10 个 worker 线程）；
- 每个线程执行 _process_item(item)，内部会调用一次 run_one(...)：
- run_one 里做翻译（可选）→ 检索 → 构造 prompt → 调 llm.generate(...) → 解析输出 → 返回一个字典；
- 多个样本的 run_one 是并发进行的，也就是同一个进程里多个线程同时在跑检索 + 调 LLM。
- 5）主线程统一写文件，避免并发写冲突
- _process_item 只负责“算结果，返回 row 字典”；
- 主线程 for fut in as_completed(...) 里拿到 row 才调用 write_jsonl_append(out_path, row)：
- 所以同一时刻只有主线程在写 jsonl，不会出现多线程同时写一个文件导致行混乱或损坏。

它对 GPU / 速度的理论效果

- 理论上：
- 如果 llm.generate 内部会释放 GIL 且允许多个并发请求命中 GPU（例如底层是 C++ 调用、能 pipeline / 多流），
- 那么多个线程会同时在 GPU 上跑不同样本，GPU 利用率和吞吐量都会上去。
- 实际上你这里提升有限的原因（之前也提过）：
- 很多本地 LLM 封装在同一进程中会串行处理 generate 调用（内部有锁 / 队列），于是线程并行变成“排队调用”；
- 评测里检索 + 等待的比例不低，单条样本的 GPU 负载不重，所以即使有一点重叠，也很难把 GPU 拉满。

总结一句：eval.num_workers 控制的是“同一进程里开多少个线程并行跑 run_one”，通过 ThreadPoolExecutor 实现，计算并发、写文件串行；能否真正提高 GPU 利用率，取决于底层 llm.generate 是否支持在一个进程内高并发地调 GPU。

设计基于多线程的并行评测，通过 eval.num_workers 配置控制并行度，切分问题集并行。

代码块

```
CUDA_VISIBLE_DEVICES=0 python src/eval_sft/eval_run.py --config configs/sft/sft_part1.yaml &
CUDA_VISIBLE_DEVICES=0 python src/eval_sft/eval_run.py --config configs/sft/sft_part2.yaml &
CUDA_VISIBLE_DEVICES=0 python src/eval_sft/eval_run.py --config configs/sft/sft_part3.yaml &
CUDA_VISIBLE_DEVICES=0 python src/eval_sft/eval_run.py --config configs/sft/sft_part4.yaml &
```

```
CUDA_VISIBLE_DEVICES=0 python src/eval_sft/eval_run.py --config configs/sft/sft_part5.yaml &

wait
```

更新评测标准：

检索类

Recall@K：gold_citations 是否出现在 retrieved topK 里

MRR@K：gold_citations 第一次出现的位置

> 这俩只看 retriever，不看 LLM，帮你区分“检索问题”还是“生成问题”。

引用类（RAG最关键）

CitePrec（引用精确率）：模型输出的 citations 中，有多少属于 gold_citations

CitationSupport（支撑率）：输出中有引用的样本里，引用是否真的来自检索证据集合（防止编 chunk_id）

> CitePrec = “引用对不对”；Support = “引用是不是瞎写的”。

拒答类

RefusalRate：模型拒答比例

RefusalF1：该拒答（gold_citations 为空）时拒答了吗？不该拒答时乱拒答了吗？

质量/效率

HallucinationRate（幻觉率）：不该拒答却拒答、或引用为空但给出事实性回答

代码块

```
python src/dpo/eval_run.py \
  --test data/dpo/qa_raw_test.jsonl \
  --llm_cfg configs/rag.yaml \
  --model_path /root/autodl-tmp/models/models--unsloth--DeepSeek-R1-Distill-Qwen-7B-bnb-4bit/snapshots/5c46879548e720d7b7d640251a09926b92627fcd \
  --lora_sft /root/autodl-tmp/LLaMA-Factory/saves/DeepSeek-R1-7B-Distill/lora/trainsft_2026-03-02-22-49-08 \
  --index_dir data/index \
  --out_dir runs

python src/dpo/eval_run1.py \
  --test data/dpo/qa_raw_test.jsonl \
  --llm_cfg configs/rag.yaml \
  --model_path /root/autodl-tmp/models/models--unsloth--DeepSeek-R1-Distill-Qwen-7B-bnb-4bit/snapshots/5c46879548e720d7b7d640251a09926b92627fcd \
```

```
  --lora_sft /root/autodl-tmp/LLaMA-Factory/saves/DeepSeek-R1-7B-Distill/lora/trainsft_2026-03-02-22-49-08 \
  --lora_dpo /root/autodl-tmp/LLaMA-Factory/saves/DeepSeek-R1-7B-Distill/lora/traindpo_2026-03-04-14-24-59
  --index_dir data/index \
  --out_dir runs
```

代码块

```
python src/dpo/eval_matrics.py \
  --test data/dpo/qa_raw_test.jsonl \
  --runs_dir runs \
  --chunks data/processed/chunks/chunks.jsonl \
  --settings sft \
  --out_csv tables/sft_v3.csv \
  --out_md tables/sft_v3.md


rm -f tables/judge_cache/deepseek_judge_cache.json
python src/dpo/eval_matrics.py \
  --test data/dpo/qa_raw_test.jsonl \
  --runs_dir runs \
  --chunks data/processed/chunks/chunks.jsonl \
  --settings baselocal,sft,dpo \
  --use_deepseek_judge \
  --judge_model deepseek-chat \
  --judge_workers 8 \
  --judge_acc_thres 0.5 \
  --out_csv tables/sft_v4.csv \
  --out_md tables/sft_v4.md

 python src/dpo/eval_matrics_answer_only_v7.py \
  --test data/dpo/qa_raw_test.jsonl \
  --runs_dir runs \
  --settings baselocal,sft,dpo \
  --use_clean \
  --use_deepseek_judge \
  --judge_model deepseek-chat \
  --judge_workers 8 \
  --judge_acc_thres 0.5 \
  --dump_judge_debug tables/judge_debug_answer_only.jsonl \
  --out_csv tables/answer_only_sft_judge.csv \
  --out_md tables/answer_only_sft_judge.md

```

```
 python src/dpo/eval_matrics_answer_only_v7.py \
  --test data/dpo/qa_raw_test.jsonl \
  --runs_dir runs \
  --settings sft \
  --use_clean \
  --ans_sim_thres 0.65 \
  --out_csv tables/answer_only_sft.csv \
  --out_md tables/answer_only_sft.md

python src/dpo/eval_matrics_answer_only_v9.py \
  --test data/dpo/qa_raw_test.jsonl \
  --runs_dir runs \
  --settings baselocal,sft,dpo \
  --use_clean \
  --emb_model_path /root/autodl-tmp/models/bge-small-en-v1.5 \
  --device cuda \
  --acc_thres 0.5 \
  --out_csv tables/answer_only_v9.csv \
  --out_md tables/answer_only_v9.md
```

C. 强化学习：用 DPO 把“引用正确、拒答正确”优化出来

用偏好优化把「引用正确 + 拒答正确 + 不胡说」这三个能力系统性优化出来

因为 PPO 需要奖励函数，训练不稳定容易发散，我选择用DPO，它通过对比学习形式直接优化偏好数据

1 数据生成

直接用 DeepSeek API 批量生成偏好数据（DPO pairs）

SFT：让模型学习“RAG 助手行为模板”——严格基于检索证据回答、按指定格式输出引用、证据不足时稳定拒答。

DPO：在相同 prompt 下学习偏好排序——更偏好“引用正确、无证据外延/幻觉、拒答策略正确”的回答，从而提升引用可靠性与拒答一致性（降低 Hallucination / 提升 CitePrec）。

代码块

```
python src/dpo/build_dpo_dataset.py \
  --llm_cfg configs/rag.yaml \
  --chunks data/processed/chunks/chunks.jsonl \
  --out_dir data/dpo \
  --n_train 2200 \
  --n_test 100 \
  --refuse_ratio 0.15
```

代码块

```
python - <<'PY'
import json
from transformers import AutoTokenizer

data_path = "/root/autodl-tmp/LLaMA-Factory/data/my_dpo_train.jsonl"  # 你实际训练用的jsonl
model_path = "/root/autodl-tmp/models/models--unsloth--DeepSeek-R1-Distill-Qwen-7B-bnb-4bit/snapshots/5c46879548e720d7b7d640251a09926b92627fcd"

tok = AutoTokenizer.from_pretrained(model_path, use_fast=True, trust_remote_code=True)

lens = []
for line in open(data_path,'r',encoding='utf-8'):
    x=json.loads(line)
    prompt = x["prompt"]
    chosen = x["chosen"]
    rejected = x["rejected"]
    # DPO 通常会拼 prompt + answer 来算长度，这里用 chosen 代表最坏情况
    t = tok(prompt + chosen, add_special_tokens=False).input_ids
    lens.append(len(t))

lens.sort()
print("N =", len(lens))
print("min/median/p90/max =", lens[0], lens[len(lens)//2], lens[int(len(lens)*0.9)], lens[-1])
PY
```

数据格式如下(prompt, chosen_response, rejected_response)：

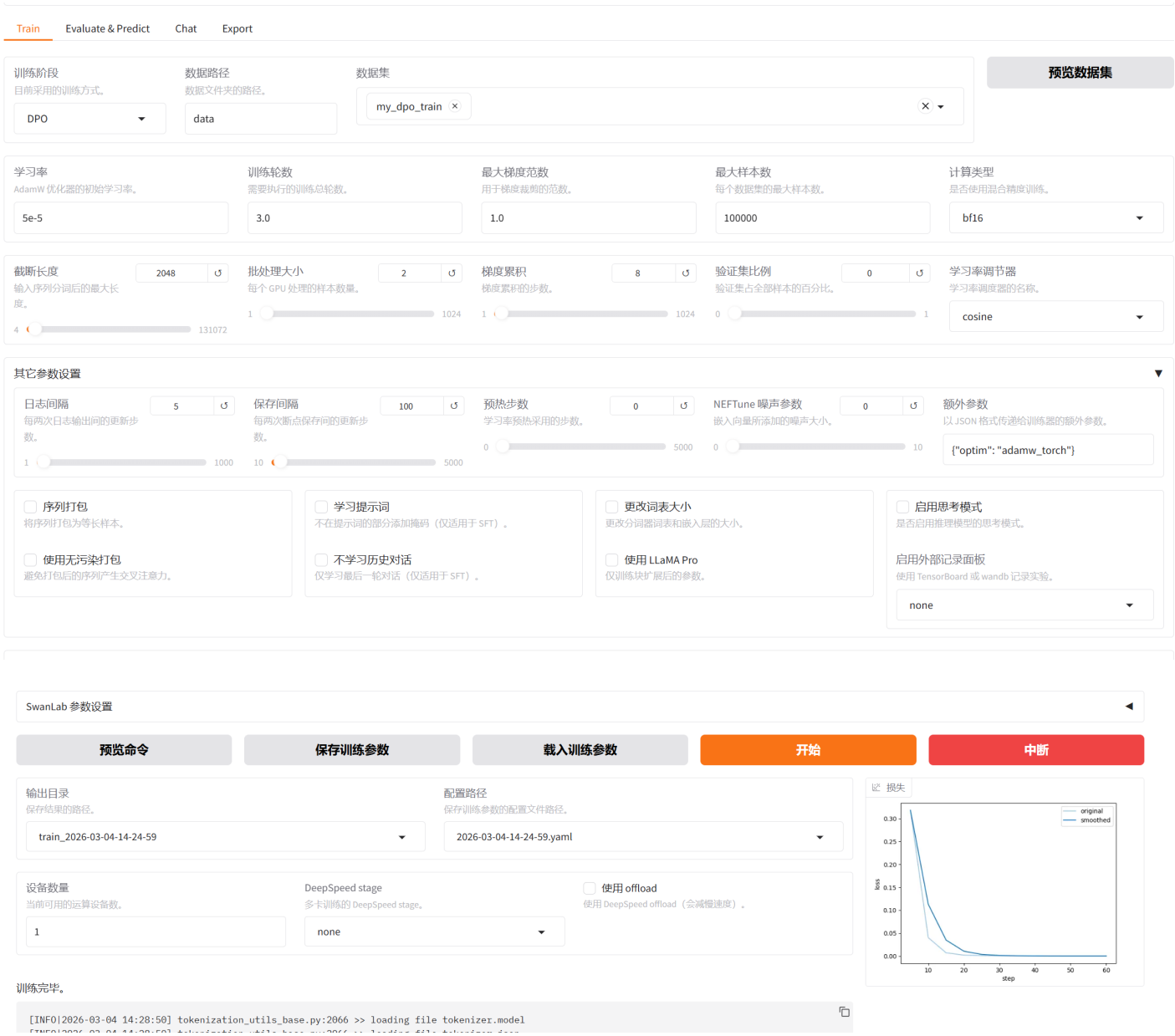
代码块

```
{
    "conversations": [
        {
            "from": "human",
            "value": "你是一个严格的RAG助手。只能使用【证据chunks】中的信息回答。\n- 如果证据不足以支持问题：必须拒答，并说明缺失点。\n- 不允许编造，不允许使用常识补全。\n- 输出末尾必须给出 citations: [\"chunk_id\", ...]，且只能引用证据chunks中出现过的chunk_id。\n\n问题：根据图8(a)，在严格、单次通过和多次通过条件下，工作流级别的成功率分别是多少？\n\n【证据chunks】\n[arxiv:2511.07392_p9_c0] 9 Fig. 7. Stage-level accuracy results: (a) Overall stage-level accuracy; (b) Stage-level accuracy by task agent type; (c) Overall accuracy of action determination components;
```

(d) Action determination component accuracy by task agent type; Error bars indicate 95% confidence intervals. Fig. 8. Workflow-level sucess rates results: (a) Overall results; (b) Results by task agent type; Error bars indicate 95% confidence intervals. 2) Workflow-level Success Rate: The workflow-level SR improves progressively from strict to single-pass to multi-pass conditions as shown in Fig. 8 (a). The SRstrict remains at 65.8%, primarily limited by transcription errors in the STT. In contrast, the SRsingle-pass ...\n\n请回答，并在末尾输出 citations: [\"chunk_id\", ...]"
        }
    ],
    "chosen": {
        "from": "gpt",
        "value": "缺少图8(a)及其相关数据。"
    },
    "rejected": {
        "from": "gpt",
        "value": "异性和疗效[63]. 该研究的结论是，基于\n工程细菌的癌症疗法值得进一步探索[61].\n\n[附录]\n附录一\n证据等级(牛津循证医学中心2011版)\n(临床)问题\n步骤1\n步骤2\n步骤3\n步骤4\n步骤5\n(等级1*) (等级2*) (等级3*) (等级4*)\n这个疾病有多普遍? (患病率)\n当地的，当前的随机样本调查(或普查) 与当地情况相匹配调查的系统综述 * * *

```
                "overall": -0.7,
                "cite_prec": 0.0,
                "cite_in_prompt": 0.0,
                "refusal_correct": 0.0,
                "overcite": 0.0,
                "missing_cit": 1.0
            }
        },
        "gap": 1.7,
        "chosen_pred_citations": [],
        "rejected_pred_citations": [],
        "chosen_pred_refusal": true,
        "rejected_pred_refusal": false
    }
}
```

2 DPO训练



3 测评

代码块

```
python src/dpo/eval_matrics_answer_only_v9.py \
  --test data/dpo/qa_raw_test.jsonl \
  --runs_dir runs \
  --settings dpo \
  --use_clean \
  --emb_model_path /root/autodl-tmp/models/bge-small-en-v1.5 \
  --device cuda \
  --acc_thres 0.50 \
  --out_csv tables/answer_only_v9.csv \
  --out_md tables/answer_only_v9.md
```

D. 具备工具调用能力的可治理 Agent 系统

背景：单轮RAG只能回答静态问题，复杂任务需要多部决策，多工具协作。而之前的步骤是：用户问题-检索-LLM-回答

Agent的作用是：让模型可以思考-调用工具-在思考-在调用-直到回答的流程。

在原有 RAG 基础上，我将 RAG 封装为工具，并引入 ReAct Agent 机制，使模型可以根据问题自主决定是否调用知识库，从而实现更复杂的推理与多步决策。

我将原本的单轮 RAG 系统升级为可工具调度架构，实现了从“检索增强生成”到“可执行知识代理”的结构演进。

方法：

运行：

代码块

```
python src/agent/agent_cli.py
```

代码块

```
🧩 Agent CLI (type 'exit' to quit)
```

```
Building prefix dict from the default dictionary ...
Loading model from cache /tmp/jieba.cache
Loading model cost 4.516 seconds.
Prefix dict has been built successfully.

👤 You: (23+7)*4/2

🤖 Agent: (23+7)*4/2 = 60
[{'llm_output': 'Thought: The user is asking for a simple arithmetic calculation: (23+7)*4/2. I should use the calculator tool.\n\nAction: calculator\nAction Input: (23+7)*4/2', 'tool_name': 'calculator', 'tool_input': '(23+7)*4/2', 'observation': '[calculator] 60.0'}, {'llm_output': 'Thought: The calculator returned 60.0. I can now provide the final answer.\n\nFinal Answer: (23+7)*4/2 = 60'}]

```

2 链接飞书

首先新建机器人获取webhook，然后写个程序feishu_webhook用于推送消息，并在命令行设置webhook 环境变量。

代码块

```
export FEISHU_WEBHOOK="https://open.feishu.cn/open-apis/bot/v2/hook/1ea975d7-e7a2-4019-b5c0-5dfcf7476826"
```

代码块

```
from integrations.feishu_webhook import feishu_send_text, feishu_send_card

feishu_send_text("✅ ProdRAG-Agent 已上线：这是一条测试消息")
feishu_send_card(
    title="ProdRAG-Agent 测试卡片",
    md="**Hello** 飞书！\n\n- 我可以把评测结果/回答推送到群里\n- 下一步可接 RAG + web_search"
)
```

Rag Agent 机器人 | 通过webhook将rag agent结果返回群

✅ ProdRAG-Agent 已上线：这是一条测试消息

ProdRAG-Agent 测试卡片

Hello 飞书！

- 我可以把评测结果/回答推送到群里
- 下一步可接 RAG + web_search

🤖 **RAG Agent 回答**

用户问题：
什么是药物注射

Agent回答：
药物注射（Drug Injection）是指使用注射器将药物直接注入人体组织或血管的给药方法。这是一种常见的医疗操作，能使药物快速进入血液循环或局部组织，起效迅速，生物利用度高。

19:34

主要类型包括：

1. **皮内注射**：注入表皮与真皮之间，常用于皮试（如青霉素皮试）。
2. **皮下注射**：注入皮下组织，常用于胰岛素、疫苗等。
3. **肌肉注射**：注入肌肉组织（如臀大肌、三角肌），适用于需缓慢吸收的药物或油剂。
4. **静脉注射**：直接注入静脉，药物立即进入血液循环，起效最快，常用于急救或输液。
5. **其他**：如关节腔内注射、鞘内注射等特殊途径。

注射给药需要严格无菌操作，由专业医护人员执行，以避免感染、损伤组织或发生空气栓塞等风险。具体药物的注射方式、剂量和部位需遵医嘱。

（注：以上信息基于一般医学知识，如需具体医疗指导，请咨询专业医护人员。）

遇到问题

调用 DeepSeek API 时的网络/服务端超时，在长时间批量评测（200 条）里必然会遇到。

📜

1. **DeepSeek 服务端偶发慢响应**
   - 模型排队、负载高
2. **长连接中途被 TLS/代理掐掉**
3. **你的网络到日本/海外节点抖了一下**

4. **单次请求 prompt 较长（chunk 多）**

👉 **这不是异常情况，而是“现实情况”**
👉 所以评测脚本**必须设计成：失败可重试、进度可恢复**

方案核心思想（面试官版）

> **评测是“长任务”，必须允许：**
>
> - 单条失败
> - 自动重试
> - 不中断整体流程
> - 已完成的不重跑
>
> 离线评测是一个长时间任务，我在 eval 脚本里做了断点续跑：
>
> 先从已有的 runs 文件里读取已完成的 qid，
>
> 在循环中自动跳过，
>
> 这样即使中途超时或中断，也可以直接恢复，不需要重跑。

如何指定路径下载hf大模型

📜 hf download deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B \
--local-dir /home/zps/models/llm/DeepSeek-R1-Distill-Qwen-1.5B

使用本地模型时遇到检索结果混杂 + 生成不受约束 → 幻觉拼接

👤 User:根据资料，哪篇论文是关于大语言模型与代码安全的系统性文献综述?

🤖 Assistant:The following generation flags are not valid and may be ignored: ['temperature', 'top_p']. Set `TRANSFORMERS_VERBOSITY=info` for more details.

Okay, so I have this query about a literature review on a specific topic. Let me try to break it down step by step.

First, the user is asking for a JSON response that includes whether a paper is a refusal, the answer, and the citations. The topic is about a system-of-systems approach to studying the interaction between large language models and code security.

Looking at the provided materials, I see that there are several PDFs and references. The first PDF, [1], is about the system of systems approach to studying the interaction between large language models and code security. It's a 2024 paper. The second PDF, [2], is about HIV-1 latent infection and its progression. The third is about LEWIS CA, MARGOLIS DM, BROWNE EP. New concepts in therapeutic manipulation of HIV-1 transcription and latency: latency reversal versus latency prevention. The fourth is about WU T, WU, WU, WU

📜 当前版本通过 Prompt 和规则约束实现 RAG QA，

后续计划通过强化微调将“基于资料、拒绝幻觉”的行为内化到模型中。

本地模型回答不太理想， Refusal / Citation / Answer 行为完全失控存在严重 hallucination、乱码、结构崩坏，大量 answer = 空字符串——base / sft 输出很乱、citation 为空、甚至编造

显存不够

在用 bitsandbytes 4bit 量化加载模型时，Transformers/Accelerate 发现显存不够，于是把一部分模块“自动分发”到了 CPU 或磁盘（offload），但 4bit 量化器不允许这种默认的分发方式，所以直接报错退出。

解决方法：4bit 时强制整模型上单卡（cuda:0）

项目讲解——Project: 医学文献 RAG 系统

S · 场景（Why）

- 使用场景：
  - 面向医学文献的问答与科普生成
- 核心问题：
  - 单路 dense 检索在专业术语上召回不稳定
  - 生成模型容易“编造引用”
- 约束：
  - 单卡 RTX 4070
  - 推理延迟需 < 15s

T · 任务（What）

- 我的责任：

    - 设计并实现可复现的 RAG 检索 + 精排 + 生成流水线
- 目标指标：
    - Recall@10 ⩾ 0.70
    - CitePrec ⩾ 0.90
    - RefusalRate 可控

A · 行动（How）

- 检索层：
    - Dense：bge-small-zh / en
    - Sparse：BM25
    - 合并策略：score normalize + merge topK
- 精排层：
    - bge-reranker-base
    - 分析 rerank_topk 对性能/延迟影响
- 生成层：
    - LLM 抽象为 Provider 接口（API / Local HF / LoRA）
- 评测：
    - YAML 驱动离线 runs
    - 自动 Error Case Analysis

R · 结果（Outcome）

- 指标：
    - Recall@10：0.73
    - CitePrec：0.955
    - AvgLatency：13.1s
- 工程结论：
    - rerank_topk 是关键超参
    - 双路检索对中文医学文本收益显著